BENCHMARK OPERATIONS

One MySQL server, six services, one gateway

Per-user queries joining across six microservice databases on one MySQL server, through QueryAPIGate: about 1.2 ms added a call, 580-780 calls a second per worker, a ping we didn't know we were paying for, why lookups want one thread, and the isolation gap to close before you rely on table allow-lists.

25 October 2026 · 6 min read

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=one-mysql-six-services. This copy was made on 4 November 2026.

A common shape for a growing system is one MySQL server, with a database per microservice: `crm` for customers, `orders`, `billing`, `inventory`, and so on. Sooner or later, something needs to read across them: a support tool, a partner API, an AI agent. "Show me this user's latest order, with the items and their payments" is three services' data in one query. MySQL can join across databases on the same server, so the question is what putting a governed gateway in front of that costs.

We measured it. We loaded TPC-C's data into MySQL 8.4 (600,000 customers, 6 million order lines) and split its tables into six databases, one per service. Then we ran six per-user queries through QueryAPIGate and straight into MySQL, side by side. The full write-up, with every table and its ranges, is Performance on MySQL.

A fixed 1.2 ms

QUERY	MYSQL DIRECTLY	THROUGH THE GATEWAY	ADDED
One customer by id	0.03 ms	1.16 ms	1.1 ms
TPC-C Order-Status (customer, last order, its lines)	0.08 ms	1.40 ms	1.3 ms
TPC-C Stock-Level (a range and a `COUNT(DISTINCT)`)	0.42 ms	1.64 ms	1.2 ms

These are single calls at p50, with the metadata cache on. The gateway adds about the same 1.1-1.3 ms to every query, so the cost is fixed, not a percentage. For a query MySQL answers in microseconds, that cost is most of the call. For one that takes 20 ms, it's noise.

The cost is the gateway's work: identity, the grant check, the SQL guard, the record of the run, and the HTTP response. All of it is Python. One worker serves 580-780 of these calls a second, while MySQL, on two cores, could answer 8 to 110 times more. So you scale with workers. At eight workers, the heaviest query kept MySQL busy on 2.5 of its 4 hardware threads, and the lighter ones on less than one.

Three round trips, one of them a ping

We counted every statement MySQL received: two per call, the query and a `ROLLBACK` when the pooled connection goes back to the pool. That's half what PostgreSQL gets, because the MySQL driver sets read-only mode and the time limit once per connection, not once per query.

Then we timed calls at 0, 0.5, 1 and 2 ms of simulated network delay. The slope of latency against delay counts the round trips a call actually waits for, and it said **three**.

The third turned out to be in the MySQL client library. Every time code asks mysql-connector for a cursor, the library pings the server first, and our driver asked for a new cursor on every query. A ping isn't a statement, so the log never showed it. Across a 0.5 ms network, that's 0.6 ms a call we can take back by reusing a cursor. It's on the list for the next release.

Lookups want one thread

THREADS PER WORKER	1	2	4	8
One customer by id (0.03 ms)	**865/s**	823	722	676
Stock-Level (0.42 ms)	630	**772**	683	642

The Docker image runs eight threads per worker. For a 0.03 ms lookup, one thread was 28% faster than eight: extra threads only compete for Python's interpreter lock. The 0.42 ms query wanted two, because with one thread a worker sits idle while MySQL works. The rule is simple: the faster the query, the fewer threads it needs. Slow reports and exports are the opposite, and that's what the eight threads are for.

Busy users make a response cache pay

Real traffic isn't uniform. A few accounts call far more than the rest. We drew users from a Zipf distribution and gave the Order-Status query a 30-second response cache (`cache_ttl: 30`):

- **59% of calls were answered from the cache,** throughput rose 23%, and MySQL got 58% fewer statements.
- **With uniform users, the same cache cost 8%.** Only 5% of 600,000 equally likely users came back within 30 seconds, and storing the rest bought nothing.

So put `cache_ttl` on queries whose callers repeat themselves, such as dashboards and popular accounts, and not on lookups every user makes once.

The store across the network

QueryAPIGate reads three rows of its own on every call: the API key, the saved query and the connection. On one instance with its store on local SQLite, those reads are cheap in time but not in CPU: a fifth of the worker's. Several instances share a PostgreSQL store instead, and then the reads cross the network too:

QUERYAPIGATE'S STORE	ONE CALL
SQLite, local	3.5 ms
PostgreSQL, 0.5 ms away	6.1 ms
PostgreSQL, 0.5 ms away, with the metadata cache	**3.4 ms**

The metadata cache (`QUERYAPIGATE_METADATA_CACHE_TTL`) keeps those three rows in memory. It caches rows, never decisions: whether a key is active, expired or used from an allowed address is checked on every call. For a multi-instance deployment, it's the difference between the second row and the third. On a single instance, it still adds 12–15% throughput.

What fills the disk

Every call is written to the store's event log, which keeps 24 hours of events for clients that resume from where they left off. In five hours of this benchmark, that came to 10.6 million events and 6 GB of SQLite: about 570 bytes a call. At 500 calls a second, the log levels off around 25 GB. Either size the store's disk for it, or shorten `QUERYAPIGATE_EVENTS_RETENTION_HOURS`.

The gap to close first

Six services in one server raise a question the performance numbers don't answer: can a key for one service be kept out of another's tables? We checked. A key allowed to read `customer` could read `crm.customer`, as intended, and could also read a `billing.customer` table in another service's database. On MySQL, QueryAPIGate's table allow-list matches bare table names; only Trino connections match the full `catalog.schema.table`.

Until that changes, keep services apart with MySQL's own grants. Give each service a MySQL user that can only see its own databases, add it as its own QueryAPIGate connection, and grant keys that connection only. Matching `schema.table` on MySQL is planned for the next release. Saved queries aren't affected: a key granted named queries can only run their SQL, which the publisher wrote.

What's next

From this run, for 0.43:

1. **Reuse a cursor per pooled MySQL connection,** to drop the ping: 3 round trips down to 2.
2. **Match `schema.table` in MySQL table allow-lists,** to close the isolation gap.
3. **Stop re-reading settings from the environment on every call:** 3.6% of the CPU.
4. **Look at autocommit for read-only MySQL connections,** which would drop the `ROLLBACK` too, and make a call a single round trip.

Check it yourself

```shell
pip install -e ".[server,mysql,postgres]" py-spy
python benchmarks/tpcc_mysql.py --out /tmp/qag-tpcc --go-tpc ./go-tpc setup A B teardown
python benchmarks/tpcc_mysql_report.py /tmp/qag-tpcc/results.jsonl
```

The harness starts its own MySQL, loads the data with go-tpc, splits it into the six databases, pins each part to its own cores, and compares releases side by side, with five rounds per comparison. It's TPC-C's data and two of its queries, but not the TPC-C benchmark, so these aren't TPC-C results. Plug the laptop in first.